

0.76.3. Un déclencheur en (0, 0) : le carré plus rapide

Le 0.76.2 avec un seul nombre changé : le B fait un pas toutes les 5 images au lieu de 15. Son carré va trois fois plus vite.

1. Ce qu'il faut comprendre

C'est le 0.76.2, avec un seul changement : `if (bimages == 5)` au lieu de 15.

Ce qui est nouveau ici : la vitesse du chronomètre. Un pas toutes les **5 images**, c'est 12 pas par seconde, **trois fois plus vite**. Le tour de 16 pas dure alors moins d'une seconde et demie.

Plus le nombre est petit, plus ça va vite, comme pour toutes les vitesses vues jusqu'ici : c'est le temps d'attente entre deux pas.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2387 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;    // le A part de (10, 0)
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;
```

3. UNE VARIABLE — ACTIF

```
uint8_t actif = 0;    // 0 : pas encore ; 1 : le A est passé par (0, 0)
```

4. UNE VARIABLE — BX

```
uint8_t bx = 10;    // le B : sa colonne...
```

5. UNE VARIABLE — BY

```
uint8_t by = 8;    // ...et sa ligne. Il apparaît au milieu, (10, 8).
```

6. UNE VARIABLE — BPAS

```
uint8_t bpas = 0;    // où il en est dans son tour : du pas 0 au pas 15
```

7. UNE VARIABLE — BIMAGES

```
uint8_t bimages = 0;    // le chronomètre du B, en images
```

8. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

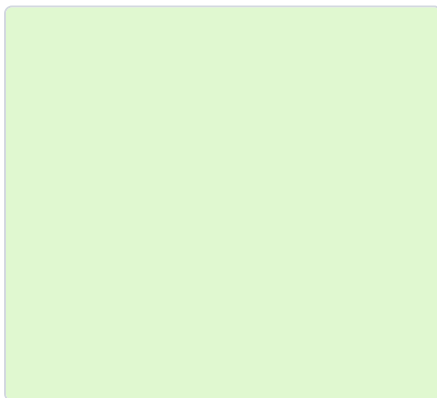
```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[0], 250); // le A suit la croix
        nombre(0, 17, x);                      // sa position, en direct
        nombre(4, 17, y);

        // Le déclencheur : le A arrive en (0, 0), et actif vaut encore 0.
        if (x == 0 && y == 0 && actif == 0) {
            actif = 1;                          // une seule fois : ensuite actif vaut 1
            poser(10, 8, ALPHABET[1]);          // le B apparaît au milieu
        }

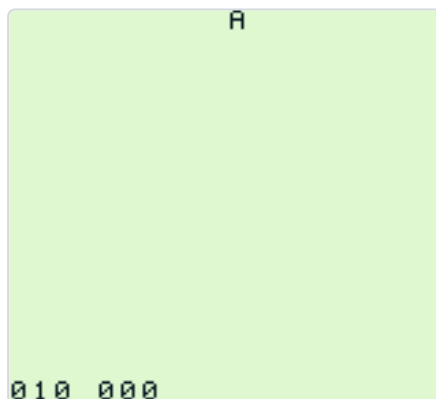
        // Les mouvements, une fois le déclencheur passé.
        if (actif == 1) {
            bimages++;                          // le chronomètre avance
            if (bimages == 5) {                 // toutes les 5 images : un pas
                bimages = 0;
                if (bpas < 4) { un_pas(bx, by, ALPHABET[1], 1, 0); } // pas 0 à 3 : à droite
                else if (bpas < 8) { un_pas(bx, by, ALPHABET[1], 0, 1); } // pas 4 à 7 : en bas
                else if (bpas < 12) { un_pas(bx, by, ALPHABET[1], -1, 0); } // pas 8 à 11 : à gauche
                else { un_pas(bx, by, ALPHABET[1], 0, -1); } // pas 12 à 15 : en haut
                bpas++;
                if (bpas == 16) { bpas = 0; } // le tour est fini : on recommence
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le même carré qu'au 0.76.2, mais trois fois plus rapide.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ en 120 images, le B a déjà fait un tour entier

Ensuite : **0.76.4. Un déclencheur en (0, 0) : deux B** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++